

1. 제어시스템의 Guidance & Control(Controller)의 각 역할 조사

Guidance: 직역하면 안내, 유도라는 뜻입니다. 제어시스템이 어디로 이동해야 하는지에 대해 안내해주고 유도해준다는 의미를 갖습니다. 후술하겠지만, 경로생성과 경로추종이 모두 Guidance에 포함될 수 있다고 생각합니다. 현재 위치를 토대로 목표점까지의 경로를 정해주는 역할을 합니다.

Control: 앞서 서술한 Guidance에서 지정한 목표점을 실제 물리적 제어를 통해 구현하는 역할을 합니다. 대표적으로 PID 시스템이 있습니다. 제어 시스템 알고리즘을 통해 속도, 자세, 위치 등 여러 부분에서 발생할 수 있는 오차를 최소화하여 시스템이 최종 목표에 잘 도달하도록 제어하는 역할을 합니다.

2. 지상차량 자율주행을 위한 Path Generation(경로생성) 기법 조사

-> Dijkstra, RRT

Dijkstra: 그래프 기반의 최단 경로 알고리즘인 것으로 조사하였습니다. 그래프를 통해 모든 경우의 비용과 경로를 분석하기에, 늘 최적해가 보장된다는 장점이 있습니다. 하지만 연산량이 많고 복잡한 고차원 공간에서는 비효율적일 수도 있다는 단점을 갖습니다.

RRT: 랜덤 샘플링을 기반으로 분석하는 기법으로 조사하였습니다. 모든 경우의 수를 다 탐색하는 것이 아닌, 시작 지점에서 무작위로 경우를 샘플링하며 경로를 탐색하기에 고차원 공간에서도 진행 속도가 빠르다는 장점을 갖습니다. 하지만 늘 최적해가 보장되지 않는 단점을 갖습니다.

처한 상황과 경우에 맞게 적절한 기법을 사용해야 한다고 생각합니다.

3. 지상차량 자율주행을 위한 Path Tracking(경로추종) 기법 조사

-> Stanley Method, Pure Pursuit

Stanley Method: 차량의 진행 방향과 목표 궤적 사이의 오차, 즉 교차 측면 오차와 방향 오차를 동시에 보정하는 기법인 것으로 조사하였습니다. 조향까지 조정할 수 있기에 곡선 주행 성능이 뛰어나다는 장점이 있지만, 시스템 제어 과정에서 오버슈팅이 발생할 수 있다는 단점이 있습니다.

Pure Pursuit: 차량의 전방 거리에 임의 목표점을 설정하고, 해당 지점을 향해 궤적을 설정하며 따라가는 기법인 것으로 조사하였습니다. 구현이 매우 직관적이고 좁은 공간에서 저속으로 주행 시 효과적이라는 장점이 있지만, 설정 거리가 너무 짧으면 진동이 발생하고 너무 길면 경로를 이탈할 수 있다는 단점이 있습니다.

주행 코스와 필요 속도에 따라 적절한 기법을 선택하여야 합니다.

4. Python에서 if문, while문, class문 사용방법을 조사, 숙지

If문: 현재 상황을 정해진 조건에 맞게 평가하여 진행합니다. '만약에 ~라면'이라는 조건에 상황을 대입하여, 해당 조건이 참일 때만 코드를 실행합니다.

조건이 참이면 if문 바로 아래에 있는 코드를 실행하고,

조건이 거짓이면 해당 코드를 건너뛰고 다음 코드로 넘어갑니다.

<예시>

학점 = 3.5

if age >= 4.0:

 print("공부를 열심히 합니다.")

else:

 print("노는 걸 좋아합니다.")

위 코드를 실행하면 조건에 해당하지 않으므로 "노는 걸 좋아합니다"를 출력하게 됩니다.

While문: 조건이 참이라면 계속해서 같은 코드를 반복 실행합니다. 조건에 해당하여 참이라면 코드를 실행하고, 코드가 모두 실행되면 처음으로 돌아가 다시 조건을 확인합니다. 이 과정을 거짓에 해당될 때까지 반복합니다. 이 때 반복을 멈추게 하는 조건을 꼭 입력해야 무한 반복 루프가 발생하지 않게 됩니다.

<예시>

count = 0

while count < 5:

 print("시제실 파이팅!")

 count = count + 1

위 코드에서도 1씩 더하다가 5를 초과하게 되면 해당 while문이 중단됩니다.

Class문: Class)는 객체를 만들기 위한 설계도 역할을 합니다. 즉, 대상의 변수와 함수를 하나의 틀로 정의하는 방법입니다. 클래스를 정의하면 이를 기반으로 실제 객체를 생성할 수 있으며, 같은 클래스에서 만들어진 객체들은 같은 구조와 기능을 공유합니다.

<예시>

```
class Student:

    def __init__(self, name, grade):

        self.name = name        # 학생 이름

        self.grade = grade      # 학생 학점

    def study(self):

        if self.grade >= 4.0:

            print(self.name, "는 이미 열심히 공부하고 있습니다.")

        else:

            print(self.name, "는 더 열심히 공부해야 합니다.")
```

클래스 사용하기

```
student1 = Student("손흥민", 3.5)

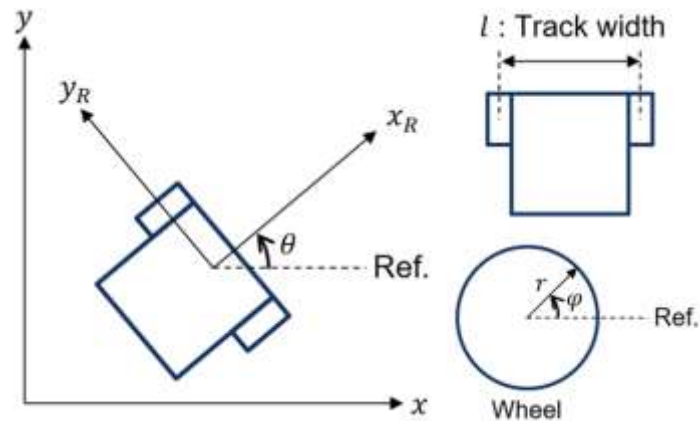
student2 = Student("이강인", 4.2)

student1.study()    # 손흥민은 더 열심히 공부해야 합니다.

student2.study()    # 이강인은 이미 열심히 공부하고 있습니다.
```

위 코드에서는 Student라는 클래스를 정의하고, __init__()을 통해 학생 이름과 학점을 초기화했으며, study() 메서드를 통해 학생 상태에 따른 메시지를 출력하였습니다. Class문은 현실 세계의 개체를 코드로 모델링하는 방법이라고 정리할 수 있습니다.

5. 지상차량 자율주행 실험을 위한 Turtlebot의 주행 개요는 다음과 같다.



x, y : Global current position of the turtlebot

θ : Angle between local and global (Robot orientation)

$\dot{\theta}$: Angular speed of robot orientation

v : Linear speed of the turtlebot

l : Track width (바퀴간거리)

r : Wheel radius

φ_1, φ_2 : Angle position of the left and right wheel (Encoder Angle)

실험에 사용하는 Turtlebot의 simplified dynamics는 다음과 같을 때, 주어진 State-space equations을 작성하세요.

$$v = \frac{1}{2}r\dot{\varphi}_1 + \frac{1}{2}r\dot{\varphi}_2$$

$$\dot{\theta} = \frac{r\dot{\varphi}_1}{l} - \frac{r\dot{\varphi}_2}{l}$$

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{\varphi}_1 \\ \dot{\varphi}_2 \end{bmatrix} = \begin{bmatrix} \square & \square \\ \square & \square \\ \square & \square \\ \square & \square \\ \square & \square \end{bmatrix} \begin{bmatrix} v \\ \dot{\theta} \end{bmatrix}$$

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \square & \square \\ \square & \square \\ \square & \square \end{bmatrix} \begin{bmatrix} \dot{\varphi}_1 \\ \dot{\varphi}_2 \end{bmatrix}$$

$$\dot{x} = v \cos \theta, \dot{y} = v \sin \theta$$

$$\Rightarrow \begin{bmatrix} v \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \frac{r}{2} & \frac{r}{2} \\ -\frac{r}{l} & \frac{r}{l} \end{bmatrix} \begin{bmatrix} \dot{\varphi}_1 \\ \dot{\varphi}_2 \end{bmatrix} = G \begin{bmatrix} \dot{\varphi}_1 \\ \dot{\varphi}_2 \end{bmatrix}, \begin{bmatrix} \dot{\varphi}_1 \\ \dot{\varphi}_2 \end{bmatrix} = \begin{bmatrix} \frac{1}{r} & -\frac{1}{2r} \\ \frac{1}{r} & \frac{1}{2r} \end{bmatrix} \begin{bmatrix} v \\ \dot{\theta} \end{bmatrix} = G^{-1} \begin{bmatrix} v \\ \dot{\theta} \end{bmatrix}$$

$$\Rightarrow \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{\varphi}_1 \\ \dot{\varphi}_2 \end{bmatrix} = \begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \\ \frac{1}{r} & -\frac{1}{2r} \\ \frac{1}{r} & \frac{1}{2r} \end{bmatrix} \begin{bmatrix} v \\ \dot{\theta} \end{bmatrix}, \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \frac{r}{2} \cos \theta & \frac{r}{2} \cos \theta \\ \frac{r}{2} \sin \theta & \frac{r}{2} \sin \theta \\ -\frac{r}{l} & \frac{r}{l} \end{bmatrix} \begin{bmatrix} \dot{\varphi}_1 \\ \dot{\varphi}_2 \end{bmatrix}$$